



CATEGORÍA: MANEJO DE MEMORIA Y DIRECTIVAS

● FÁCIL

1. **Reserva mixta:** Diseña un programa que reserve en memoria (desde 0x10010000): 2 words, 3 bytes consecutivos, un espacio alineado de 12 bytes, y finalmente 5 words más.
2. **Cadena con datos:** Define en memoria la cadena "MIPS R2000" seguida de un byte 0xFF y luego una palabra con valor 0x12345678. Asegúrate de que la palabra esté correctamente alineada.
3. **Vector de bytes:** Crea un vector de 10 bytes con los valores del 1 al 10 en decimal. Luego reserva espacio para una palabra que será la suma de todos ellos.

● MEDIO

4. **Matriz 4x4:** Define una matriz de enteros 4x4 almacenada por filas con los números del 1 al 16. Incluye después un espacio para guardar la suma de la diagonal principal.
5. **Estructura de datos:** Simula una estructura de datos "Persona" que contenga: un byte para edad, un word para ID, una cadena de 20 caracteres para nombre (usa .space), y otro word para sueldo.
6. **Reorganización de memoria:** Define 4 bytes consecutivos con valores 0xAA, 0xBB, 0xCC, 0xDD. Luego reserva espacio para reorganizarlos como: una word en orden inverso, y otra word intercambiando los pares (0xBB, 0xAA, 0xDD, 0xCC).

● DIFÍCIL

7. **Array de estructuras:** Crea un array de 3 "registros" donde cada registro tiene: 1 word (código), 2 bytes (cantidad), 1 word alineado (precio). Asegúrate de que todos los words estén correctamente alineados.



CATEGORÍA: CARGA Y ALMACENAMIENTO



FÁCIL

8. **Swap simple:** Dados dos words en memoria, intercambia sus valores usando solo registros temporales.
9. **Copia de vector:** Define un vector de 4 words con valores 10, 20, 30, 40. Cópialos en orden inverso a otra posición de memoria.
10. **Extracción de bytes:** Dada la palabra 0xAABBCCDD, extrae cada byte individual y guárdalos en 4 posiciones consecutivas de memoria.



MEDIO

11. **Empaquetamiento:** Dados 4 bytes en posiciones consecutivas de memoria, empaquéalos en una sola word respetando little-endian y guárdala en memoria.
12. **Rotación de bytes:** Dada una palabra 0x12345678, crea otra palabra rotando sus bytes una posición a la izquierda (resultado: 0x34567812).
13. **Half-words cruzados:** Dadas dos palabras A=0x11223344 y B=0x55667788, crea una palabra C tomando el half-word alto de A y el half-word bajo de B.



DIFÍCIL

14. **Transposición de matriz 2x3:** Define una matriz 2x3 por filas. Genera su transpuesta (matriz 3x2) en otra zona de memoria usando cargas y almacenamientos apropiados.
15. **Compresión de booleanos:** Tienes 32 bytes en memoria, cada uno con valor 0 o 1. Comprímelos en una sola word donde cada bit represente el valor del byte correspondiente.



CATEGORÍA: OPERACIONES ARITMÉTICAS



FÁCIL

16. **Promedio de 4 números:** Dados 4 enteros en memoria, calcula su promedio (suma/4) usando desplazamientos para la división.
17. **Conversión Celsius a Fahrenheit:** Dado un valor en Celsius, calcula Fahrenheit usando la fórmula $F = C \times 9/5 + 32$. Puedes aproximar 9/5 como multiplicar por 2 menos 10%.
18. **Valor absoluto:** Dado un entero con signo, guarda su valor absoluto en otra posición de memoria.



MEDIO

19. **Suma de cuadrados:** Dados tres números a, b, c, calcula $a^2 + b^2 + c^2$ sin usar mult (usa sumas repetidas optimizadas con desplazamientos cuando sea posible).
20. **División con resto personalizado:** Dados dos números A y B, calcula cociente y resto, pero si el resto es mayor que B/2, ajusta el cociente sumando 1 y recalcula el resto negativo.
21. **Potencia de 2:** Dado un entero N (entre 0 y 31), calcula 2^N usando desplazamientos y guárdalo en memoria.



DIFÍCIL

22. **MCD (Máximo Común Divisor):** Implementa el algoritmo de Euclides para calcular el MCD de dos números usando restas o módulo.
23. **Fibonacci hasta N:** Genera los primeros N números de la secuencia de Fibonacci y guárdalos en un vector en memoria.
24. **Multiplicación de matrices 2x2:** Dadas dos matrices 2x2, calcula su producto y guárdalo en memoria.



CATEGORÍA: OPERACIONES LÓGICAS Y DE BITS



FÁCIL

- 25. **Conteo de bits a 1:** Dado un word, cuenta cuántos bits están en 1 y guarda el resultado.
- 26. **Paridad:** Determina si un número tiene paridad par o impar (cantidad de bits en 1) y guarda 0 o 1.
- 27. **Máscara de bits:** Dado un número, pon a 1 los bits en las posiciones 3, 7, 15 y 20, sin modificar los demás.



MEDIO

- 28. **Inversión de bits específicos:** Dado un word, invierte solo los bits en posiciones pares (0, 2, 4, 6, ..., 30).
- 29. **Extracción de campo:** De un word, extrae los bits 12-19 (8 bits) y guárdalos alineados a la derecha en otro word.
- 30. **Rotación de bits:** Implementa una rotación a la izquierda de 5 bits de un word (los bits que salen por la izquierda entran por la derecha).



DIFÍCIL

- 31. **Compresión run-length simple:** Dado un byte con patrón de bits, identifica la mayor secuencia consecutiva de 1s y guarda su longitud.
- 32. **Palíndromo binario:** Verifica si la representación binaria de un número es un palíndromo (se lee igual de izquierda a derecha que de derecha a izquierda).



CATEGORÍA: CONTROL DE FLUJO



FÁCIL

33. **Máximo de 5 números:** Dado un vector de 5 enteros, encuentra el máximo usando comparaciones y saltos condicionales.
34. **Conteo condicional:** Dado un vector de 10 enteros, cuenta cuántos son mayores que 50.
35. **Búsqueda lineal:** Dado un vector de 8 enteros y un valor objetivo, determina si el valor está en el vector (guarda 1 si está, 0 si no).



MEDIO

36. **Selection Sort:** Implementa el algoritmo de ordenamiento por selección para un vector de 7 enteros.
37. **Suma de pares:** Dado un vector de N enteros, suma solo los números pares y guarda el resultado.
38. **Inversión de string:** Dado un string en memoria, inviértelo en otra posición (último carácter primero, etc.).



DIFÍCIL

39. **Búsqueda binaria:** Dado un vector ordenado de 16 enteros y un valor objetivo, implementa búsqueda binaria para encontrar su posición (retorna -1 si no está).
40. **Insertion Sort optimizado:** Implementa insertion sort para un vector de 10 enteros, pero detén el proceso si en alguna pasada no haces ningún intercambio (ya está ordenado)



CONSEJOS PARA RESOLVER:

- **Fácil:** Enfócate en usar correctamente las instrucciones básicas y el direccionamiento
- **Medio:** Requieren combinar múltiples conceptos (bucles + aritmética, lógica + condicionales)
- **Difícil:** Necesitan algoritmos más complejos, optimización y manejo cuidadoso de múltiples registros



COMBINACIÓN DE CONCEPTOS AVANZADOS



FÁCIL

41. **Suma acumulativa:** Dado un vector de 6 números, crea otro vector donde cada posición contenga la suma acumulativa hasta ese punto. Ej: $[1,2,3,4] \rightarrow [1,3,6,10]$
42. **Verificador de rango:** Dado un número N, verifica si está en el rango $[10, 50]$. Guarda 1 si está en el rango, 0 si no.
43. **Contador de negativos:** Dado un vector de 8 enteros con signo, cuenta cuántos son negativos.
44. **Espejo de nibbles:** Dado un byte 0xAB, inviértelo a 0xBA intercambiando los nibbles (4 bits superiores con 4 bits inferiores).



MEDIO

45. **Factorial iterativo:** Calcula el factorial de N (donde $N \leq 12$ para evitar overflow) usando un bucle.
46. **Filtrado de vector:** Dado un vector de 10 enteros, crea otro vector que contenga solo los números positivos, y guarda la cantidad de elementos copiados.
47. **Compresión de espacios:** Dado un string con múltiples espacios consecutivos, genera otro string donde los espacios múltiples se reduzcan a uno solo.
48. **Número Armstrong:** Verifica si un número de 3 dígitos es Armstrong (ej: $153 = 1^3 + 5^3 + 3^3$). Guarda 1 si lo es, 0 si no.
49. **Mediana de 5 números:** Dados 5 números, encuentra la mediana (el valor del medio después de ordenarlos).
50. **Conversión binario a decimal visual:** Dado un número, genera un string que represente sus 8 bits menos significativos como caracteres '0' y '1'.



DIFÍCIL

51. **Merge de dos vectores ordenados:** Dados dos vectores ordenados ascendentemente de tamaño 5 cada uno, crea un tercer vector ordenado de tamaño 10 con todos los elementos.
52. **Substring search:** Dado un string largo y uno corto (patrón), encuentra la primera posición donde aparece el patrón (retorna -1 si no existe).
53. **Números primos en rango:** Genera todos los números primos entre 2 y 50 usando el algoritmo de la Criba de Eratóstenes simplificado.
54. **Evaluación de expresión postfija simple:** Dada una expresión en notación postfija con números de un dígito (ej: "23+4*"), evalúala usando una pila simulada en memoria.



CATEGORÍA: MANIPULACIÓN AVANZADA DE BITS



MEDIO

- 55. **Gray Code:** Convierte un número binario a su código Gray equivalente (cada bit es XOR del bit actual con el anterior).
- 56. **Contar transiciones:** Dado un word, cuenta cuántas veces cambia el valor de un bit al siguiente (transiciones $0 \rightarrow 1$ o $1 \rightarrow 0$).
- 57. **Aislamiento de bit más significativo:** Dado un número, crea otro que tenga solo el bit 1 más a la izquierda activado. Ej: $0b0010110 \rightarrow 0b0010000$
- 58. **Swap de bits:** Dado un número, intercambia los bits en posiciones i y j (por ejemplo, posiciones 3 y 15).



DIFÍCIL

- 59. **Potencia de 2 más cercana:** Dado un número N , encuentra la potencia de 2 mayor o igual más cercana. Ej: $20 \rightarrow 32$, $16 \rightarrow 16$
- 60. **Representación de flotante:** Simula la estructura de un float IEEE 754 simplificado: extrae el signo (1 bit), exponente (8 bits) y mantisa (23 bits) de un word y guárdalos por separado.



CATEGORÍA: DESAFÍOS ALGORÍTMICOS



MEDIO (pero desafiantes)

61. **Eliminación de duplicados:** Dado un vector de 12 enteros, crea otro vector sin elementos duplicados y guarda su tamaño final.
62. **Segundo mayor elemento:** Encuentra el segundo elemento más grande en un vector de 10 enteros (sin ordenar todo el vector).
63. **Suma de dígitos iterativa:** Dado un número, suma sus dígitos. Si el resultado tiene más de un dígito, repite hasta obtener un solo dígito. Ej: $9875 \rightarrow 29 \rightarrow 11 \rightarrow 2$
64. **Verificador de paréntesis balanceados:** Dado un string con paréntesis '(' y ')', verifica si están balanceados correctamente.



DIFÍCIL

65. **Quicksort parcial:** Implementa solo la partición del algoritmo Quicksort: dado un vector y un pivot, reorganiza el vector con elementos menores a la izquierda y mayores a la derecha.
66. **LCM (Mínimo Común Múltiplo):** Dados dos números, calcula su MCM usando la fórmula: $MCM(a,b) = (a \times b) / MCD(a,b)$
67. **Conversión de string a entero (atoi):** Dado un string que representa un número decimal (ej: "1234"), conviértelo al valor entero equivalente.
68. **Rotación de vector:** Dado un vector de N elementos, rótalos K posiciones a la izquierda de manera eficiente (sin usar vector auxiliar completo).



CATEGORÍA: CASOS ESPECIALES Y EDGE CASES



MEDIO

- 69. **Manejo de overflow en suma:** Suma dos números grandes y detecta si hay overflow. Si hay, guarda un flag en memoria indicándolo.
- 70. **División segura:** Implementa una división que maneje el caso especial de división por cero (guarda 0xFFFFFFFF como resultado de error).
- 71. **Normalización de vector:** Dado un vector de enteros, encuentra el máximo y mínimo, y crea un nuevo vector con valores normalizados entre 0 y 100.



DIFÍCIL

- 72. **Multiplicación sin mult:** Implementa la multiplicación de dos números usando solo sumas y desplazamientos (algoritmo de multiplicación binaria).
- 73. **División sin div:** Implementa división entera usando solo restas y comparaciones.
- 74. **Raíz cuadrada entera:** Calcula la raíz cuadrada entera de un número usando el método de búsqueda binaria o Newton-Raphson simplificado.



CATEGORÍA: PROBLEMAS PRÁCTICOS



MEDIO

- 75. **Checksum simple:** Calcula un checksum de un vector de bytes sumando todos y tomando el complemento a 2 del resultado truncado a 8 bits.
- 76. **Compresión RLE completa:** Implementa Run-Length Encoding completo: dado un vector de bytes, crea otro con pares (valor, cantidad). Ej: [5,5,5,7,7,8] → [5,3,7,2,8,1]



DIFÍCIL

- 77. **Calculadora de expresiones:** Evalúa una expresión aritmética simple (solo +, -, , /) dada como string con números de un dígito. Ej: "3+42"
- 78. **Generador de permutaciones:** Dado un pequeño conjunto de 4 números, genera todas sus permutaciones posibles en memoria.
- 79. **Algoritmo de Luhn:** Implementa el algoritmo de validación de Luhn (usado en tarjetas de crédito) para verificar un número de 10 dígitos.
- 80. **Sudoku validator 4x4:** Dada una grilla de Sudoku 4x4 resuelta, verifica si es válida (sin repetir números en filas, columnas y subcuadrantes 2x2).